

Engineering-based processes and agile methodologies for software development: a comparative case study [☆]

Éric Germain, Pierre N. Robillard ^{*}

Département de génie informatique, École Polytechnique de Montréal, Université de Montréal, C.P. 6079, Succ. Centre-ville, Montréal, Qué., Canada H3C 3A7

Available online 30 April 2004

Abstract

The emergence of various software development methodologies raises the need to evaluate and compare their efficiencies. One way of performing such a comparison is to have different teams apply different process models in the implementation of multiple versions of common specifications. This study defines a new cognitive activity classification scheme which has been used to record the effort expended by six student teams producing parallel implementations of the same software requirements specifications. Three of the teams used a process based on the Unified Process for Education (UPEDU), a teaching-oriented process derived from the Rational Unified Process. The other three teams used a process built around the principles of the Extreme Programming (XP) methodology. Important variations in effort at the cognitive activity level between teams shows that the classification could scarcely be used without categorization at a higher level. However, the relative importance of a category of activities aimed at defining “active” behaviour was shown to be almost constant for all teams involved, possibly showing a fundamental behaviour pattern. As secondary observations, aggregate variations by process model tend to be small and limited to a few activities, and coding-related activities dominate the effort distribution for all the teams.

© 2004 Elsevier Inc. All rights reserved.

Keywords: Empirical software engineering; Process measurement; Cognitive activity; Productivity

1. Introduction

There is increasing interest in the teaching of software processes (see, for instance, El Emam, 2001; Halling et al., 2002; Höst, 2002; Robillard et al., 2003; Umphress and Hamilton, 2002). However, this interest has not yet translated into acceptance of a common set of process principles. Rather, two main software process approaches seem to be emerging. The first promotes the utilisation of a discipline-based engineering process involving effective definition of the roles to be played, activities to be performed and artifacts to be produced.

This approach involves the production of artifacts to support early decision-making on requirements and design matters, effective communication, knowledge reuse and mutual work inspection. The main principle here is that efforts made in up-front planning activities and in artifact production will result in lower overall cost, timely product delivery and better software quality. The Rational Unified Process (RUP) (Kruchten, 2000) is an example of a process that fits this approach. The Unified Process for Education (UPEDU) is a RUP-based process designed for teaching the software processes in software engineering and computer science programs. The process is built on a set of focused activities with significant cognitive content, and on a set of essential artifacts that should be needed for small projects. The process also defines basic roles such as “Designer”, “Implementer”, “Tester”, “Project Manager” and “Reviewer” which are easily understandable by developers who are inexperienced. Roles, activities and artifacts are grouped into disciplines, which are categorized as either engineering or management. The

[☆] An earlier version of this paper has been published with the title “What Cognitive Activities Are Performed in Student Projects?” in the Proceedings of the Sixteenth Conference on Software Engineering Education & Training (CSEET’2003).

^{*} Corresponding author. Tel.: +1-514-340-4238; fax: +1-514-340-3240.

E-mail addresses: eric.germain@polymtl.ca (É. Germain), pierre-n.robillard@polymtl.ca (P.N. Robillard).

UPEDU has been extensively described in Robillard et al. (2001, 2003) and is readily available online (École Polytechnique, 2002).

The other approach, called “Agile Software Development” (Agile Alliance, 2003; Cockburn, 2002), promotes quick response to changes in requirements as well as extensive and ongoing collaboration between the development team and the customer. The approach specifically downplays the importance of formal processes and comprehensive documentation (Agile Alliance, 2003). It is based on the assumption that one cannot truly anticipate project requirements right at the beginning of a software development project, and that the proper way to deliver timely, quality software in a cost-effective manner is instead to build flexibility within the development activities. The Manifesto for Agile Software Development (Agile Alliance, 2003) provides the basic values for agile development in detail. Some methodologies inspired by this approach include Adaptive Software Development (Highsmith, 2000), Scrum (Rising and Janoff, 2000), the Crystal family (Cockburn, 2002; Highsmith, 2002), Feature-Driven Development (Palmer and Felsing, 2002), Dynamic System Development Method (Stapleton, 1997) and Extreme Programming (XP; see Beck (1999)).

There is no such thing as a universal process or methodology. Organizations will typically develop their own process or methodology based on the approaches described, and often on an existing process model. For example, an organization may use tools provided with the RUP package to build a custom process containing a chosen subset of the proposed activities and artifacts. Alternatively, an organization may simply take the generally recognized set of XP practices and adapt them to its particular context.

Meanwhile, students enrolled in a software engineering program who have received training on software processes are expected to be, at the end of the program, more sensitive to issues affecting software quality, cost and life cycle. This does not mean, however, that those individuals will actually apply everything they have learned. Previous studies (Germain et al., 2002a,b; Robillard, 1998) in the context of the Software Engineering Studio, a project-oriented course for senior-level students, have shown a significant gap between theory as taught and practice. In these studies, “effort slips” were used as an indicator of relative activity intensity. Analyses performed were, however, limited by the activity and artifact classification of the UPEDU-based process used, making it rather difficult to determine exactly which elemental tasks had been performed.

Using those studies as a foundation, we defined a set of cognitive activities aimed at recording the various activity states of a software developer during the course of a project. The utilisation of such a classification allows us to study the impact of software process notions

learned on the cognitive activities actually performed by the students.

An important benefit of such an approach is that it opens the door to comparative studies of the two approaches. Advocates of these are keen to debate the respective merits of each, or even sometimes the fact that there is any dichotomy at all between them (see, for instance, Antón, 2003; Beck and Boehm, 2003; Wells, 2003). Several studies have been performed to analyse the value of a particular practice (see, for instance, Cockburn and Williams, 2001). However, very few studies have compared two complete software process approaches. This paper presents a case study of the implementation of our cognitive activity classification to the comparative analysis of two process models, one based on the UPEDU, the other based on XP.

These observations should not be readily generalized to industrial practices because of the academic nature of the setting and the impact of the particular project, life cycle and technology chosen. Besides, repeating such an experiment in an industrial setting would be quite difficult because of the need to record individual cognitive activities at the developer level. However, we think that the study presented in this paper provides clues which may be useful when evaluating the importance of a specific software engineering process.

2. The Software Engineering Studio

The Software Engineering Studio is an optional project-oriented course offered to senior-year students in computer engineering at the École Polytechnique de Montréal. Its purpose is to allow students to gain practical experience in software development by participating in a small-scale software development project. Teams of students must develop a complete implementation based on software requirements specifications provided by the instructors. Also, they must use a well-defined software engineering process. Participants thus acquire experience early on in building an entire operational software project through design, implementation, testing and management activities. This project course teaches them the realities of teamwork, milestones and time constraints. As a secondary objective, students become more familiar with a specific application domain or set of technologies. An earlier version of the Studio is presented in detail in Robillard (1996). The Studio has also served as a test bed for the study of development effort and artifact quality. Some individual studies performed using data generated in a Studio have been documented in Germain et al. (2002a,b), Robillard (1998). Several other software engineering programs implement other paradigms of the concept of a Studio. For instance, Blake and Cornett’s Software Engineering II class (Blake and Cornett, 2002) implemented a setting

that was aimed at teaching every step of a typical software development cycle to students split into specialised teams in the context of a single, class-wide project. By contrast, in our approach, students work in small teams, each developing their own version from common specifications. Also, in our setting, students are free to organize their work however they wish, as long as they deliver the required artifacts at each defined milestone.

The Winter 2002 Studio featured the development of a Web-based meeting management system aimed at organizers of meetings where the number and geographic dispersion of participants make scheduling difficult. The software system to be developed would allow meeting coordinators to send availability requests to a set of individuals, so that each of them can specify personal availability periods. The set of availability periods would then be graphically represented using a special calendar tool to enable a coordinator to visualise the relevant information at a glance, making the scheduling decisions easier. The decisions can then be transmitted electronically to all participants. The soft-

ware system would be responsible, among other things, for ensuring proper data storage and updates, as well as for communications among all participants. All communications would be performed using standard e-mail. All the students had to use identical software requirements specifications and were instructed not to add any additional features not explicitly requested. Fig. 1 shows a screenshot from one of the software products delivered.

The main feature of the Winter 2002 Studio was the use of two different software engineering processes. Although all the teams were competing on the same project, each team would follow one of the two proposed software development approaches. One of the goals of the study was to determine the influence of the software process on the participants' behaviour. Thus, three of the teams were assigned a process based on the UPEDU. The other three teams were assigned a process built around the Extreme Programming methodology.

Fig. 2 illustrates the prescribed software development schedule. The top row shows the timeline of iterations

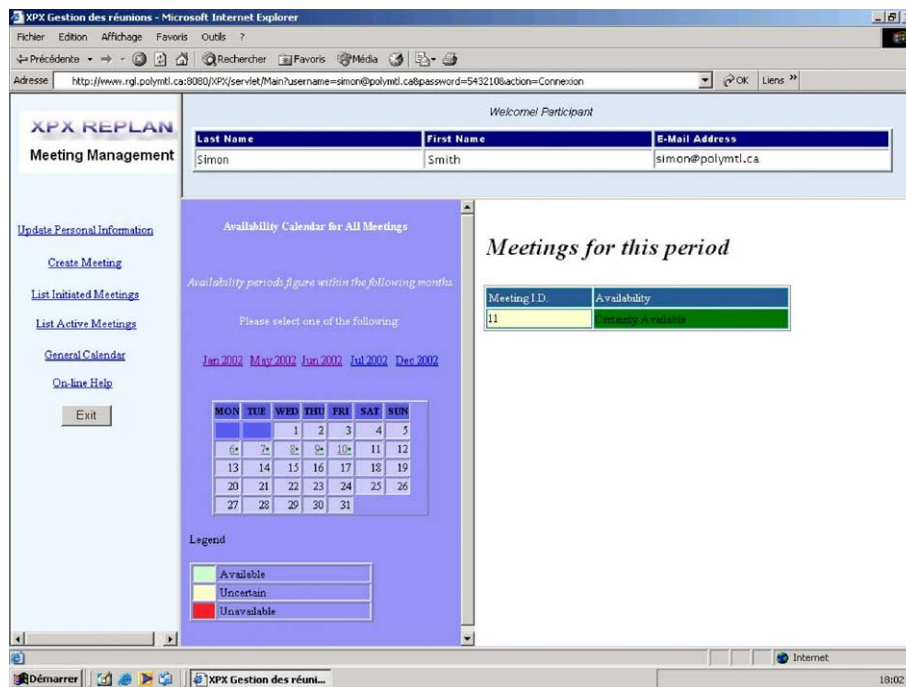


Fig. 1. Screenshot from a completed product.

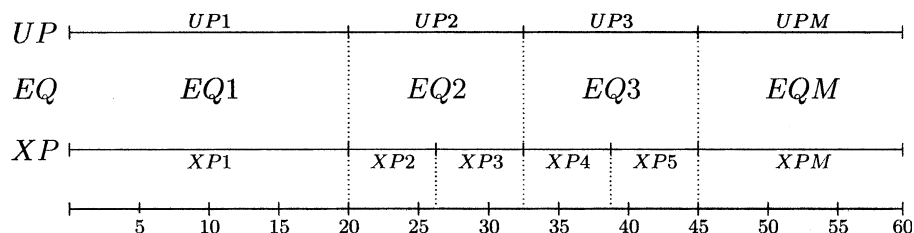


Fig. 2. Software development schedule (in working days).

for UPEDU, while the bottom row shows the same information for XP. The lower scale represents the nominal number of academic calendar days since the beginning of the project. For example, iterations UP3 and XP5 both finished on the 45th day of the project. The middle row illustrates the timing of the equivalent iterations that had been defined for data analysis purposes and which are explained below.

A common release-level framework was used to define the development schedule for both processes. Thus, for all the teams, an initial specifications document was provided at the beginning of the semester. A complete implementation of that specification was due after 45 days. Thereafter, a second specifications document was issued which requested modest architectural changes to the system. Implementation of those changes was due following a further 15-day period. Iterations XP1 through XP5 and UP1 through UP3 belong to the initial development cycle, while iterations XPM and UPM belong to the maintenance phase. An important element to mention is that no specific constraints were imposed for specific iterations, other than the requirement to deliver specification-compliant end-products and linked artifacts at the end of the development cycle. In other words, all activities could be performed at various levels within all iterations. Thus, teams had complete freedom to schedule development work throughout the development cycle. Evaluating workload and risks as well as scheduling development work are part of the tasks that a software engineer must be able to perform on an autonomous basis.

The concept of iteration applied here complies with the definition found in Robillard et al. (2003):

An iteration is a mechanism that enables the interaction between the software life cycle and the software process by defining milestones that are often associated with the phases of the product. Sets of iterations are also used to define the software process.

Iterations are common to both traditional and agile processes. However, the average size of a single iteration is likely to vary depending on the process model used. Beck (2000, Glossary) defines an iteration as “a one-to-four-week period”, while the UPEDU (École Polytechnique, 2002 “Guidelines: Software Development Plan”) considers iterations of less than one month as short and “... more suitable for the Construction phase, where the degree of new functionality to be included and the degree of novelty are low”. One reason for these differences is simply that people using the UPEDU require some considerable time to produce a significant amount of documentation, while people using XP are likely to choose not to do so. At the same time, one may argue that the additional investment in artifacts should

enhance the productivity of the software engineers using UPEDU. However, this claim has not been proven and rightly constitutes one of the aspects that was studied in this setting.

We chose, therefore, to arbitrarily customise iteration lengths for each of the processes used. Thus, the development schedule for the XP process would include a larger number of iterations covering the same time frame. However, the initial iterations time frame at the beginning of the project was of particular concern to us. The first iteration is crucial for laying out the skeleton of the system. Also, the instructors wanted to leave enough time for every participant to become accustomed to the development environment provided and minimally comfortable with the language and technologies. The initial iteration has therefore been kept identical in length for both processes. Also, the maintenance cycle has been limited to a single iteration following general agreement by the students that this would be sufficient considering the limited scope of the changes requested. The remaining iterations were set out such that the ratio of XP iterations to UPEDU iterations would be 2:1. Correspondence of UPEDU iteration end-dates to XP iteration end-dates was required to facilitate analysis of the resulting data in terms of effort expended. A set of equivalent iterations, EQ1, EQ2, EQ3 and EQM, was defined for that purpose.

Students were free to team up with whomever they wished to and to register for the process model they wanted to use, provided that the following three conditions were fulfilled: We had to have five teams of four students each and one team of three students. Three of the teams had to use the UPEDU, and the other three had to use the XP-based process. The three-student team had to use the UPEDU, since three people cannot all be pair-programming at the same time. The only problem we ran into was that two students were competing for the fourth seat on one UPEDU team. We chose to break the tie by drawing a winner's name.

There was no preliminary assessment of the participants' skills, as it was assumed that all the students were equally capable of performing their tasks correctly and efficiently. However, some variations did exist among individuals. A small number of students were not yet familiarized with the UPEDU. Also, a small number of students had been exposed to the Java language and to related technologies. Nevertheless, we feel that these differences did not create any significant asymmetry among the teams.

3. Cognitive activity classification

In previous Studios, students had been asked to record effort expended under each process activity. That approach has the benefit of allowing a direct measure-

ment of the process itself. However it can only be used on the assumption that the list of activities defined in the process covers every possible work situation without generating excessive overlap.

Such an assumption has not been confirmed. Indeed, an analysis performed using data from the 2001 Studio (Germain et al., 2002a) showed the possible presence of ambiguity and confusion among participants in relation to the process activities as defined by the instructors. Another problem with the approach was that the presence of two separate software processes prevented the utilisation of a single process-based scheme that would allow comparison of the effort expended by all the teams. An alternative approach was to use a process-independent classification which would lead to the implicit assignment of effort to the proper activity. A classification based on the evaluation of explicit, mutually exclusive cognitive activities constituted an interesting path to this target.

Table 1 illustrates the classification that was used for the study. Participants were presented a total of 14 cognitive activities and asked to tag each effort slip to be recorded with one and only one of these activities. Three of these activities dealt mostly with tasks that we wished to exclude from our study, such as training and systems administration. These activities are not included in Table 1, and all effort slips recorded under them have been excluded from this study.

Although most activity names are self-explanatory, we provide a short description of them below. Two distinct groupings of cognitive activities were defined for analysis purposes. These groupings were not revealed to the participants. Each represents one dimension in Table 1. The grouping along the horizontal axis, called “step-oriented”, represents the sequence of steps typically performed in the development of a software product. The grouping along the vertical axis, called “state-oriented”, illustrates the cognitive states likely to be taken during software development.

The “Active” state occurs when a developer is performing any activity the purpose of which is to generate or modify an artifact. It includes the cognitive activities

“Draw” and “Write”, which refer to the production of diagrams and text of all kinds respectively. It also includes a set of activities which collectively cover everything related to coding, integration and testing: “Code”, “Code & Test”, “Integrate & Test” and “Test”. This redundant labelling of activities was necessary because, under XP, these activities are often intertwined in practice. Thus, all plausible combinations of coding and testing are taken into account. Also, the “Integrate & Test” activity reflects the fact that integration is presumably a short duration activity which immediately leads to testing.

The “Reflexive” state occurs when a stand-alone developer is performing any activity which does not generate or modify any artifact. The “Browse/Search” activity refers to the action of reading documents or Web pages in a non-specific order, as when searching for documents that will eventually be read, while the “Read” activity refers to the action of reading a specific document such as a textbook or article for the purpose of assimilating a chunk of information. The “Think” activity refers to the process of self-reflection and thus encompasses every effort expended by a stand-alone participant for which no input or output exists.

The “Interactive” state refers to work performed by two or more teammates at the same time. The “Discuss” activity refers to every discussion taking place between a team member and one or more individuals who may or may not be team members. The “Inspect/Review” activity refers to the technical review activities that may be performed following the production of any artifact.

The “Preparation” step refers to activities which may be considered as pre-requisites for coding, while the “Implementation” step covers all coding, integration and testing effort. The “Control” step corresponds to explicit quality control work and includes the sole activity “Inspect/Review”.

The original classification included other activities the occurrence of which would be interpreted as merely accidental and weakly linked to the fundamental behavioural characteristics of the participants. We removed all slips declaring any of those activities from our subsequent analysis.

Table 1
Cognitive activity classification

	Preparation	Implementation	Control
Active	Draw Write	Code Code & Test Integrate & Test Test	
Reflexive	Browse/ Search Read Think		
Interactive	Discuss		Inspect/Review

4. Analysis of cognitive activities performed

4.1. Raw-data analysis

The first analysis step is to look at the raw data for each team based on the effort slips that were submitted. Fig. 3 illustrates the effort slips corresponding to the cognitive activities of each XP team (XPA, XPB, XPC), and of each UPEDU team (UPA, UPB, UPC). The state- and step-oriented categories are shown along the left and right sides of the diagram respectively. We chose

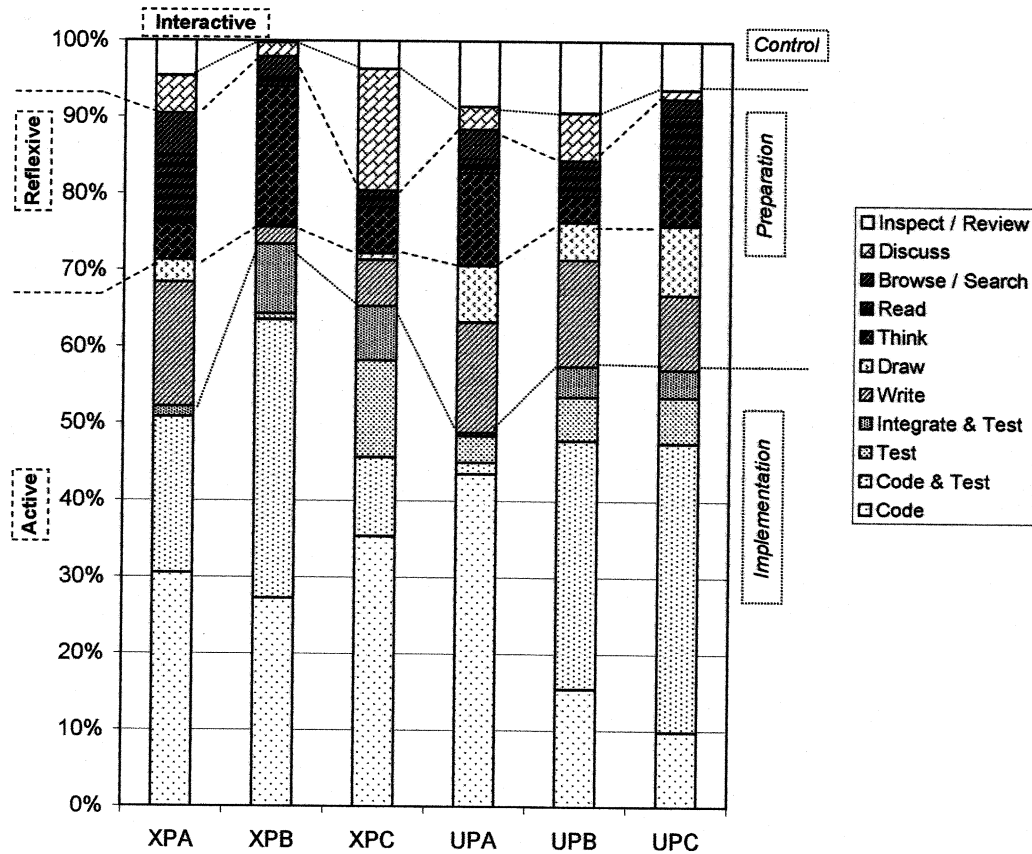


Fig. 3. Effort distribution by activity, by team.

at this point to remove all data related to the maintenance phase, which was initially set up to measure the relative maintainability of each software system. We suspect that the various design choices made by the individual teams may have had a significant impact on the effort patterns deployed, and therefore on their convergence.

Potential sources of team productivity variations are many; for instance, productivity gaps between individual developers are often very important (see, for instance, Brooks, 1995, chapter 3) and may generate even wider gaps at the team level (Cockburn, 2002, chapter 4). In order to eliminate the influence of such variations, all effort data has been normalized relative to total effort for a given team.

Fig. 3 clearly shows wide variations in intensity among teams for each cognitive activity taken individually. This result shows that effort convergence at the cognitive activity level is low. However, relative effort expended on “active” cognitive activities (see left side of Fig. 3) falls into the 71–76% range, which is narrow. This suggests a very strong inherent behaviour characteristic which is independent of the process model used. By contrast, effort distribution between the “Reflexive” and “Interactive” categories varies widely from team to

team, and likewise shows no relation to the process model.

Implementation activities (see right side of Fig. 3) represent between 49% and 73% of total effort for each team. This clearly shows that, beyond the central analysis, design and testing skills that are, rightly, promoted within the software engineering community, this discipline remains a coding-intensive one, even when performed by students aware of the importance of software process activities. This finding might provide a part of the answer to the question raised in McConnell (2002): “How important is software construction?” Software construction is very important indeed, at least in terms of its intensity relative to other categories of activities.

The only category showing any apparent influence of the process model is the “Control” category, where all UPEDU teams expended more effort (average of 8%) than XP teams (average of 3%).

4.2. Aggregate analysis

Fig. 4 illustrates the aggregate effort distribution by activity as a percentage of total effort expended for all teams. The diagram also shows the relative amounts of effort for each state-oriented category. The “Code” and

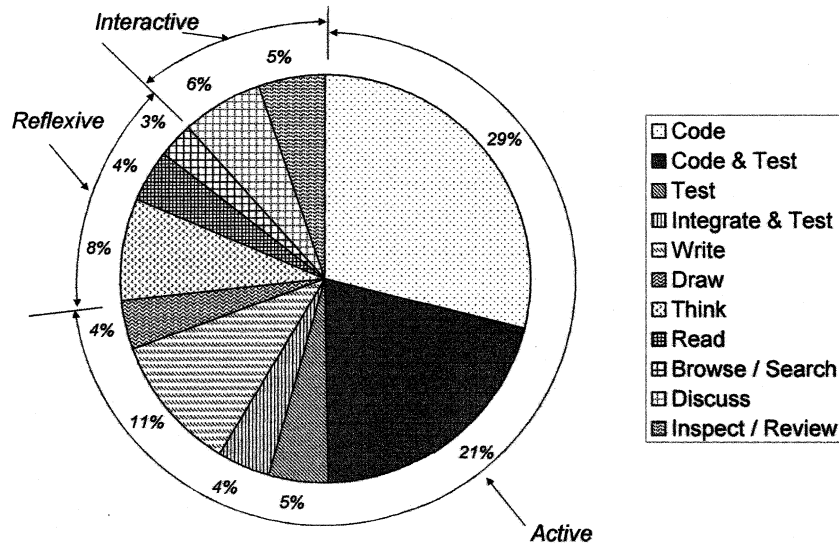


Fig. 4. Aggregate effort distribution by cognitive activity.

“Code & Test” activities have aggregate relative weights of 29% and 21%, respectively. All other activities have weights in the 3–11% range. Activities in the “Active” category include almost 75% of total effort.

Fig. 5 shows the aggregate information for each process model. The separation between state-oriented categories is shown with bold dotted lines, while the separation between the “Draw” and “Write” activities and the other activities in the “Active” category is shown with thinner dotted lines. Coding and testing activities are less intensive under the UPEDU. However, the “Draw” and “Write” activities include more effort

under this process model, bringing total effort under the “Active” category to almost identical levels for the two process models.

4.3. Inter-process analysis

Fig. 6 illustrates the relative weight of each cognitive activity in the two process models. The *process activity index* (PAI) for a given activity is defined as

$$PAI = \ln(E_{UP}/E_{XP})$$

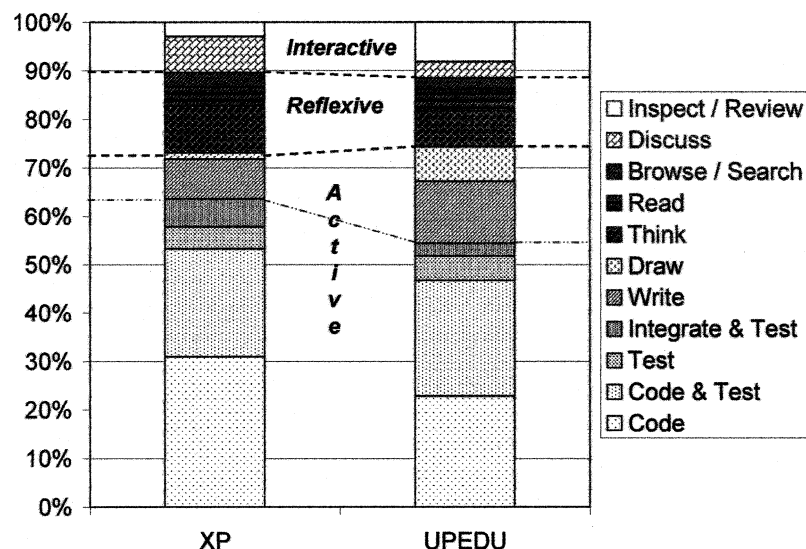


Fig. 5. Aggregate effort by cognitive activity, by process (%).

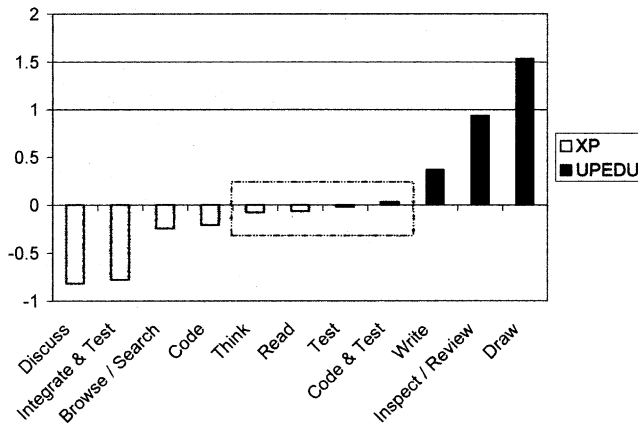


Fig. 6. Weight index by cognitive activity.

where E_{UP} and E_{XP} represent the amount of relative effort performed by the UPEDU and XP teams respectively for that activity. Thus, an index value of zero indicates identical effort for a given activity. Negative index values mean that the effort is greater for the XP teams, while positive index values mean a greater effort for the UPEDU teams. Activities are ordered by increasing index value.

Fig. 6 illustrates the essence of the cognitive activities of the two processes. Positive indices shown for the “Draw” and “Inspect/Review” activities were to be expected, since those cognitive activities, while required under a UPEDU-like process, are not covered at all by the XP methodology. Under XP, production of diagrams is not recommended by some of the principal gurus (see, for instance, Jeffries et al., 2000, Chapter 16), while inspection and revision are theoretically performed “on-line” in the course of regular pair-programming tasks. At the other end of the axis, the explicit role emphasis on the “Integrate & Test” activity reflects the intertwined nature of integration and testing under XP. That the activity is performed mainly by XP-based teams should not be considered a surprise. The presence of the “Discuss” activity at the very left end of the axis posed more of a question, since the XP methodology promotes several teamwork practices such as pair programming. It is interesting to observe that each activity in the “Interactive” category is dominant in one of the process models. This suggests a real influence of the process model on the relative intensity of these activities. The framed activities in the middle of Fig. 6 are considered to be equivalent for the two process models, since their index values are close to zero.

Fig. 7 illustrates calculation of the PAI using the state- and step-oriented categories. The “Control” category, which encompasses little total effort, is overexposed here because the index calculation is based on a simple logarithmic function. The other five categories are considered to be equivalent for the two process models.

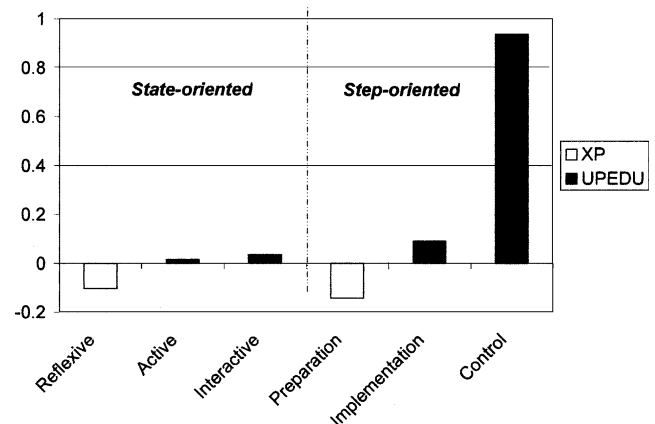


Fig. 7. Weight index by category.

5. Concluding remarks

5.1. Study limitations

Developing software is an open problem. There can be as many solutions as there are individuals or teams. In this study, all the teams provided acceptable software products in relation to the requirements specifications issued. All were also constrained by a development schedule and met all deadlines.

Even though the software process used seemed to have an impact on the importance of some cognitive activities, we did not observe any significant relation between the process used and the magnitude of the overall effort. Effort expended by the XP teams as a whole indeed exceeded effort expended by the UPEDU teams by a whopping 29%. However, the external factors that may have affected this figure are numerous and make this result highly questionable. The only three-participant team was a UPEDU team, which showed the least total effort of the six. Also, the project required quick learning of the Java Servlet technology by the participants. Since the XP teams had to start coding almost immediately, they faced technological difficulties earlier than the UPEDU teams. We observed significant technology-related knowledge transfers from the XP teams to the UPEDU teams at the time when the latter started producing code. It must be noted that other kinds of knowledge transfers, for instance those related to architectural decisions, seem not to have occurred. Traces of such transfers have not been found in the resulting artifacts, except in the form of reuse of a few key external components by more than one team. Total absence of knowledge transfer would have been very difficult to achieve in practice.

Effort performed in the context of this analysis made up the entire contribution to the realization of a frozen software requirements specifications document. Using evolving requirements in the course of a project would

bring a very interesting dimension to such a study in the future. Requirements evolution is a usual part of software development and constitutes a strong basis for the justification of the XP approach by its promoters. However, we predict that the impact of such a change on the effort mix would be quite limited unless some other major changes are implemented in the setting, such as making teams swap their respective projects after a delivery milestone. In any case, it is important to understand that responsiveness to change is highly influenced by the previous design choices made by a team, which in any case suffers from lack of experience. Measuring effort expended in such a context would therefore include the additional challenge of trying to exclude the influence of that factor, which may prove very difficult in practice.

Two important objectives are at the centre of the case for engineering-oriented processes. First, well-defined design activities and artifacts are meant to ensure that important design decisions, especially at the architectural level, are made properly and before the costs of change become too high, assuming these costs will rise exponentially as generally assumed in the “traditional” software engineering community (Pressman, 2001). Second, the value of the various artifacts that are developed through the project will be felt the most when another development team tries to maintain the software product, particularly if the original team members are not available for consultation. Unfortunately, our study neither confirms nor denies those statements. The benefits of mandatory design activities and artifacts are most evident in large projects, while the setting featured a small project. Measuring those benefits would require the realization of a similar study using a much larger project. Also, the impact of the methodology on product maintainability could only be measured through the realization of maintenance activities on software products developed using different methodologies. The Studio class as established can constitute a very suitable environment for such maintenance projects.

The validity of the study rests heavily on the level of confidence that exists in the effort slips entered by the participants. Difficult experiences from earlier editions of the Studio show that the instructors cannot wait until the end of the semester to validate the data as recorded. Such validation must occur in a continuous manner and be enforced through weekly meetings with participants. All irregular situations, for instance dubious effort slips or time periods without any entry, were signalled to the students as soon as detected, and immediate corrective action was requested in all cases. This attitude has generated two benefits. First, it helped improve the effective quality of the recorded data. Second, it gave the students the impression that the instructors really cared about effort slip quality and were determined to ensure it. This pushed the students to act proactively and pre-

vent problems by paying careful attention to their task of accurately recording the effort expended.

5.2. *Observations on the methodology*

The main goal of this paper was to define a methodology that would enable comparison of effort performed by developers at the cognitive level, so that fundamental behaviour characteristics of a development team which are independent of the process used can be identified. In this respect, the consistent balance of effort intensity between activities in the “Active” category and other activities in the state-oriented categorization seems to reflect such characteristics. Such an observation opens the door to the development of an effort prediction paradigm which would use compound program size estimation techniques encompassing a whole set of artifacts, instead of just code length, for instance. Research activities pre-requisite to this goal include validating this supposed invariance of the “Active” category, and measuring the result of restructuring the underlying activity classification. Ultimately, a useful target would be to define a cognitive activity set that is powerful enough to provide invariance at a very fine granularity level. This is far from being achieved as yet, since variability between teams at the individual cognitive activity level is very considerable. In the meantime, there might be some important benefits to testing the hypothesis of invariance of the “Active” category in a few industrial settings under various processes or methodologies, in order to establish its validity and usability in practice.

We applied the principle of effort normalization relative to total effort by teams throughout this study, and thus did not analyse in detail the relation between overall effort and its distribution by cognitive activity. Such an analysis would be useful, but would require a better understanding of the factors that influence total effort output for a given team, such as team size and schedule compression. This, in turn, would be best achieved using a larger number of teams, as well as multiple studio editions. Such a strategy would allow the identification of factors of influence coming from particular conditions.

5.3. *Participants’ behaviour*

This study illustrates, among other things, a basic observation of team software development based on two different software engineering processes. In spite of the limited scope of the study, a few general conclusions can be drawn, although these conclusions need more experimentation in order to be validated.

The effort expended on core activities within each development project is more or less independent of the

software engineering process used. Rather, one process will emphasise a particular type of activity over another. This shifted emphasis does not have a spectacular effect on the overall distribution of the cognitive activities performed. One possible interpretation is that some cognitive activities will require a minimal effort investment regardless of the software process used.

We observe that a well-defined software process such as the UPEDU will put more emphasis on the engineering aspects of the software implementation by stressing the pre-coding activities, while the XP-based process will put more emphasis on ad hoc communications. While these observations are totally in line with the definition of the processes involved, what is most interesting is that these differences between processes are simply not as great as one may have expected and have no impact on the effort-intensive coding-related activities.

We did not observe any significant relationship between the requirement for explicit artifacts for the UPEDU teams and the productivity of those teams. Informal discussions with the participants who used the UPEDU revealed that most of them did not feel the benefit of having produced those artifacts. One possible explanation is that, since the project was very small, most relevant information could be mastered easily by the participants and communicated verbally, with greater efficiency than through the time-consuming production of documents. Of course, it is possible that such an explanation may represent more of an impression than a concrete reflection of the true characteristics of developer behaviour.

5.4. Teaching software development

The main advantage of a well-defined software engineering process is that students are aware of the various activities that are likely to be needed for the proper development of a software product. They may thus decide to put more emphasis on a particular activity while at the same time being able to anticipate the impacts of their decisions. It is not clear whether XP as a whole is suitable for teaching at the undergraduate level or not. It is already recognized that XP is aimed at teams of talented and experienced individuals. Indeed, experience often comes from using a more structured approach, from which one can pick the valuable aspects and throw away the rest. XP is often learned through the pairing of an experienced programmer with a junior one. Academic environments, at least in their usual form, do not fit in well with such an approach. However, it may be important to familiarize students with the nature of XP (or another agile methodology) once they have been trained in a more classical process approach. Of course, some methods which have emerged in the light of XP, such as pair programming (see, for instance, Williams

and Upchurch, 2001), can be considered for teaching purposes independently of the process involved.

Acknowledgements

We are grateful to Mihaela Dulipovici who participated in the preparation of the project course, acted as teaching assistant for the course and was deeply involved in artifact and effort slip quality evaluation. Also, this project would not have been possible without the participation of all the students enrolled in the Software Engineering Studio course during the Winter 2002 semester. We would also like to thank Alexandre Moïse and Martin Robillard for their insightful comments while we were building the requirements specifications for the semester project.

This work was partly supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) under grant A0141.

References

- Agile Alliance Web site. 2003. Available from <<http://www.agilealliance.org>>.
- Antón, A.I., 2003. Successful software projects need requirements planning. *IEEE Software* 3, 44–47.
- Beck, K., 1999. Embracing change with Extreme Programming. *IEEE Computer* 10, 70–77.
- Beck, K., 2000. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, Boston, MA.
- Beck, K., Boehm, B., 2003. Agility through discipline: a debate. *IEEE Computer* 6, 44–46.
- Blake, M.B., Cornett, T., 2002. Teaching an object-oriented software development lifecycle in undergraduate software engineering education. In: *Proceedings of the 15th Conference on Software Engineering Education and Training (CSEET'02)*. IEEE Computer Society, pp. 234–240.
- Brooks Jr., F.P., 1995. *The Mythical Man-Month: Essays on Software Engineering*, Anniversary edition. Addison-Wesley Longman, Boston, Massachusetts.
- Cockburn, A., 2002. *Agile Software Development*. Addison-Wesley, Boston, MA.
- Cockburn, A., Williams, L., 2001. The costs and benefits of pair programming. In: Succi, G., Marchesi, M. (Eds.), *Extreme Programming Examined*. Addison Wesley, Boston, MA.
- École Polytechnique de Montréal. 2002. UPEDU. Available from <<http://www.yoopeedoo.org>>.
- El Emam, K., 2001. Software engineering process. Guide to the Software Engineering Body of Knowledge: trial version (version 1.00), No. 9. IEEE Computer Society, Los Alamitos, California.
- Germain, É., Dulipovici, M., Robillard, P.N., 2002a. Measuring software process activities in student settings. In: *Proceedings of the 2nd ASERC Workshop on Quantitative and Soft Computing Based Software Engineering (QSSE'2002)*, Banff, Alberta, Canada, pp. 44–49.
- Germain, É., Robillard, P.N., Dulipovici, M., 2002b. Process activities in a project based course in software engineering. In: *32nd ASEE/IEEE Frontiers in Education Conference*, IEEE, pp. S3G-7–S3G-12.

- Halling, M., Zuser, W., Köhle, M., Biffl, S., 2002. Teaching the Unified Process to undergraduate students. In: *Proceedings of the 15th Conference on Software Engineering Education and Training (CSEET'02)*. IEEE Computer Society, pp. 148–159.
- Highsmith, J.A., 2000. *Adaptive Software Development: a Collaborative Approach to Managing Complex Systems*. Dorset House Publisher, New York.
- Highsmith, J.A., 2002. *Agile Software Development Ecosystems*. Addison-Wesley, Boston, MA.
- Höst, M., 2002. Introducing Empirical Software Engineering Methods in Education. In: *Proceedings of the 15th Conference on Software Engineering Education and Training (CSEET'02)*. IEEE Computer Society, pp. 170–179.
- Jeffries, R., Anderson, A., Hendrickson, C., 2000. *Extreme Programming Installed*. Addison-Wesley, Boston, MA.
- Kruchten, P., 2000. *The Rational Unified Process: An Introduction*. Addison-Wesley, Reading, MA.
- McConnell, S., 2002. I Know What I Know. *IEEE Software* 3, 5–7.
- Palmer, S.R., Felsing, J.M., 2002. *A Practical Guide to Feature-Driven Development*. Prentice Hall PTR, Upper Saddle River, NJ.
- Pressman, R.S., 2001. *Software Engineering: a Practitioner's Approach*, fifth ed. McGraw Hill, Boston, MA.
- Rising, L., Janoff, N.S., 2000. The Scrum software development process for small teams. *IEEE Software* 4, 26–32.
- Robillard, P.N., 1996. Teaching software engineering through a project-oriented course. In: *Proceedings of the 9th Conference on Software Engineering Education and Training (CSEE'96)*. IEEE Computer Society, pp. 85–94.
- Robillard, P.N., 1998. Measuring team activities in a process-oriented software engineering course. In: *Proceedings of the 11th Conference on Software Engineering Education and Training (CSEET'98)*. IEEE Computer Society, pp. 90–101.
- Robillard, P.N., Kruchten, P., d'Astous, P., 2001. YOOPEEDOO (UPEDU): a process for teaching software process. In: *Proceedings of the 14th Conference on Software Engineering Education and Training (CSEET'01)*. IEEE Computer Society, pp. 18–26.
- Robillard, P.N., Kruchten, P., d'Astous, P., 2003. *Software Engineering Process with the UPEDU*. Addison-Wesley, Boston, Massachusetts.
- Stapleton, J., 1997. *DSDM, Dynamic Systems Development Method: the Method in Practice*. Addison-Wesley, Reading, MA.
- Umphress, D., Hamilton Jr., J.A., 2002. Software Process as a Foundation for Teaching, Learning, and Accrediting. In: *Proceedings of the 15th Conference on Software Engineering Education and Training (CSEET'02)*. IEEE Computer Society, pp. 160–169.
- Wells, D., 2003. Don't solve a problem before you get to it. *IEEE Software* 3, 44–47.
- Williams, L., Upchurch, R.L., 2001. In: *In Support of Student Pair-Programming, SIGCSE2001*. ACM press, pp. 327–331.

Éric Germain is a graduate student working in the Software Engineering Research Laboratory at the École Polytechnique de Montréal, where he received his B.Eng. degree in industrial engineering in 1993. He has previously held various positions in the fields of industrial engineering and R&D financing. His current research interests include software process improvement, software measurement and software engineering education. He is a licensed professional engineer in the Province of Québec, Canada, and a member of the IEEE Computer Society.

Pierre N. Robillard is a full professor and current chair of the Department of Computer Engineering at the École Polytechnique de Montréal, where he heads the Software Engineering Research Laboratory. His current research interests include cognitive aspects in software engineering, software collaborative work, software measurement, software engineering education and software processes. He received his B.Sc. degree from the Université de Montréal in 1969, an M.A.Sc. degree in computer communications from the University of Toronto in 1973 and a Ph.D. degree in electrical engineering from Université Laval in 1977. He recently co-authored the book “Software Engineering Process with the UPEDU”. He is a licensed professional engineer in the Province of Québec, Canada and a member of IEEE, ACM, EACE and CIPS.